

Keepnet

Coding Agent Build Brief

Prepared for Aran and the coding agent | 3 October 2026

Build a mobile-first fishing log that makes recording a session and each catch quick at the water. Anyone with a Google account can sign up. Personal logs stay private by default; anglers can optionally share selected catch details on a Google Maps catch map. Host the app on Cloudflare Pages.

Confirmed requirements

- **App name:** Keepnet
- **Sessions:** record the location, type of fishing and target species
- **Individual catches:** attach photos and record weight and bait type
- **Authentication:** secure Google sign-in, open to any Google user
- **Hosting:** Cloudflare Pages
- **Social map:** Google Maps with catch-photo pins, optional sharing and control over which catch details other people can see

Recommended first release

Use a responsive installable web app, with a clear active-session view and a prominent Log catch button. Support session and catch editing, ending a session, history filters, personal bests by actual species, private media storage, a shared catch map, field-level sharing controls, export and account deletion. Treat the details below as implementation recommendations unless identified as confirmed above.

Default privacy: open registration means anyone can create their own private log. Sharing is an explicit action per catch. The original log and unselected fields remain private. Show the audience and a preview before publishing.

Scope boundaries

Release 1 is online-first, with explicit unsent-draft preservation and reliable retries. Full offline synchronization, comments, followers, competitions, weather, AI fish identification, payments and native app-store builds are later work. Short videos are an optional next phase, subject to processing and cost decisions. Do not expand the first release to include them.

Instruction to the coding agent

Implement the core journey first: Google sign-in, start a session, log a catch, add a photo, end the session find it in history, then opt in to a map post that exposes only the chosen details. Prove two-user data isolation before polishing secondary screens. Use the proposed architecture unless a concrete incompatibility is found; explain any substitution and keep Cloudflare Pages as the host.

This brief authorizes a plan, not production account changes. Obtain owner approval for paid services, production deployment, new persistent credentials and account or permission changes. No production accounts, keys, domain or budget are supplied here.

Screens and core journeys

Navigation and visual direction

Use Home, Sessions, Discover and Profile as the primary navigation; put catch history within Sessions. Discover is the Google Maps catch map. Prioritize a one-handed phone layout with large controls, readable outdoor contrast, a sticky primary action and minimal typing. Keep the desktop layout responsive rather than building a separate interface.

Screen	Required behavior for the proposed MVP
Sign in	Explain that logs are private. Show Continue with Google, loading, cancelled sign-in and recoverable errors. Restore the intended screen after authentication.
Discover	Show clustered catch-photo pins and a list fallback. Tapping a pin opens only the details the poster shared. Include loading, empty-map and map-error states.
Home	Show the active session, recent sessions and a small personal summary. If there is no session, make Start session the primary action. Empty state explains the next step.
Start session	Location name required; optional coordinates/manual map pin and peg. Fishing type and one or more target species required. Start time defaults to now and is editable. Optional notes.
Active session	Show location, elapsed time, targets and catch timeline. Keep Log catch prominent. Allow session details to be edited and the session to be ended. A zero-catch session is valid.
Log or edit catch	Actual species required, with target species as quick choices plus Other/unknown. Time defaults to now. Weight, bait/lure, photos and notes can be added quickly; optional length and released/kept/unknown.
Session detail	Show duration, catch count, biggest weighed fish and photo gallery. Offer edit/delete with clear consequences. Historical catches remain editable.
History and profile	Filter sessions/catches by venue, actual species and date range. Show personal best per species. Profile contains weight units, sign out, export and account deletion.

Keep the capture flow fast

- Remember the last bait/lure within the current session; suggest it for the next catch, but let the user change it
- Offer camera and gallery input, multiple photos, a preview and remove/retry controls. Saving catch details must not depend on photo upload success
- Prefer a suggested single active session per user. If one exists, offer Resume or End session before starting another; do not silently close it

Location and time defaults

Always support manual venue entry. Ask for device location only after the user chooses Use my location; permission denial must not block logging. Use Google Maps for the social map. Venue search is optional; enable any additional Places service only after approving its setup and costs. Store event times in UTC with the session IANA timezone so trips and daylight-saving changes display correctly.

Data model and validation

Use UUIDs (or equivalent opaque identifiers) for app records. Firebase UID is the account key. All owned tables carry owner_uid for efficient scoped queries; child rows must also reference a parent owned by that same UID. The logical model is User → Session → Catch → Photo, plus reusable user-owned Locations. Keep social sharing in separate Post/SharedMedia tables (page 8), not a public flag that exposes the private Catch payload.

Entity	Suggested fields
User	uid PK; display_name; email only if needed; weight_unit (kg or lb_oz); created_at; disabled_at; min_auth_time; deletion_state
Location	id PK; owner_uid; name; latitude/longitude nullable; notes; created_at; updated_at. Coordinates and venue names are private.
Session	id PK; owner_uid; location_id nullable; location_name snapshot; coordinates snapshot nullable; peg nullable; fishing_type; started_at; ended_at nullable; timezone; notes; version; created_at; updated_at
SessionTarget	session_id; owner_uid; species_key or normalized custom label. Unique per session/species; multiple targets allowed.
Catch	id PK; owner_uid; session_id; species_key; species_label snapshot; caught_at; weight_g nullable; original_weight_value/unit nullable; bait_type; bait_notes; length_mm nullable; disposition; notes; version; created_at; updated_at
Photo	id PK; owner_uid; catch_id; generated_object_key; content_type; bytes; width/height; sort_order; status; created_at. R2 holds the bytes, not D1.
Support records	Idempotency: owner_uid, key, operation, payload_hash, result, expiry. Cleanup jobs: object_key, owner_uid, state, retry_count. Keep operational records bounded.

Validation rules

- Distinguish target species from actual caught species. Fishing-type choices can start with coarse, carp, sea, fly, predator/lure and other; use a stable vocabulary with custom labels rather than forcing one regional taxonomy
- Weight is optional: NULL means not weighed, never zero. Store canonical integer grams. For kg, accept up to three decimal places; for lb/oz, validate nonnegative pounds and ounces from 0 to under 16. Convert once with explicit rounding and preserve entered values for faithful editing
- Require positive measured weights/lengths. End time cannot precede start time. Catch time must lie within the session interval; offer to adjust the session or correct the catch rather than silently changing time
- Set server-side text, file and pagination limits. Reject unknown writable properties, invalid coordinates and foreign-user parent IDs. Treat notes as plain text

Database integrity and statistics

Use prepared SQL, foreign keys and composite uniqueness/relationships where practical to enforce same-owner ancestry. Index (owner_uid, started_at), (owner_uid, session_id, caught_at), species and location filters. Derive counts, duration and maximum weighed catch from current data; exclude NULL weights from personal bests. Preserve historical location snapshots when a saved venue changes.

Architecture and Google identity

Layer	Proposed implementation and responsibility
Client	React + TypeScript + Vite, responsive routes and a web-app manifest. This stack is recommended, not a user mandate. The client never decides authorization.
Web host	Cloudflare Pages serves the built frontend. Same-origin Pages Functions serve /api/* and run in the Workers runtime. Verify dependency compatibility early. [1]
Database	Cloudflare D1 stores relational records, ownership, idempotency and cleanup jobs. Bind it as DB in each environment. [2]
Photos	Private Cloudflare R2 bucket bound as PHOTOS. The authenticated API controls upload, read and delete; no public bucket domain or r2.dev access. [2, 3]
Identity	Firebase Authentication with Google as the only enabled sign-in provider. Firebase supplies identity; the app API enforces access to D1 and R2. [4, 5]

Sign in and verify every protected request

The client signs in through the Firebase web SDK and sends its Firebase ID token in an HTTPS Authorization: Bearer header. The API validates the token before resolving the authenticated UID. Use a maintained Workers-compatible JWT library and the documented Firebase verification rules; do not assume the full Node Admin SDK will run unchanged in Pages Functions.

Verify the allowed algorithm, signature and trusted key ID, exact project audience and issuer, expiration, issued-at/authentication times and nonempty subject. Refresh signing keys according to their cache headers. Reject tokens for other projects and arbitrary Google OAuth tokens. The UID comes only from the verified token, never the request body. [5]

Mobile login is an early technical spike

Configure the Google provider, Firebase authorized domains and exact OAuth redirect URIs for a stable staging origin and production origin. Confirm that production consent/audience settings allow users outside any development test-user list. Request only basic sign-in identity, not Gmail, Drive or other unrelated scopes.

Test Android Chrome, iPhone Safari and installed-PWA launch/login flows before building the rest of the app. Firebase redirect login can break when third-party storage is blocked. For Pages hosting, use a documented non-Firebase-hosting solution; if proxying the auth helper, transparently forward its required paths/methods rather than issuing a 302. Configure authDomain and redirect URIs together. [6]

Session revocation and deletion

A valid signature does not detect revocation. Check app disabled/deleting state on every request and use min_auth_time to invalidate older app sessions. Before launch, document and test how provider-level disabled/revoked accounts are detected and the maximum stale-token window. Reauthenticate for account deletion. Never label an unchecked token as revocation-verified. [5]

API contract and photo handling

Use a versioned API namespace such as /api/v1. All routes below require authentication. Separate shared-post routes are specified on page 8 and must return only a server-generated allowlist. Apply ownership checks to private lists, lookups, mutations, summaries, nested relations and file reads. Use 404 for a missing or foreign-owned object, without revealing that it exists.

Route group	Suggested operations
/me	GET profile; PATCH unit preference; GET /me/export; DELETE /me after recent reauthentication and explicit in-app confirmation
/sessions	GET with cursor/date/venue/species filters; POST; GET/PATCH/DELETE /sessions/:id; POST /sessions/:id/end
/sessions/:id/catches	GET or POST catches scoped to the owned parent; GET/PATCH/DELETE /catches/:id for individual records
/catches/:id/photos	POST image bytes plus bounded metadata; GET photo metadata list; GET /photos/:id/content; DELETE /photos/:id
/locations and /stats	Owned location CRUD; GET /stats for totals and personal bests, with the same date/species/venue filters as applicable

Consistent response and retry behavior

Return a stable error envelope containing code, message, field_errors when applicable, and request_id. Use 401 for missing/invalid identity, 422 for validation, 409 for version conflict, 413 for oversized files and 429 with retry guidance for limits. Do not expose SQL, object keys, tokens or stack traces.

Use client-generated idempotency keys for create/end/upload actions. Scope each key to owner and operation, bind it to a payload hash, and return the original result on replay. Reject a reused key with different content. Use a version field or ETag for edits; show a conflict and preserve the draft instead of silently overwriting a newer change.

Photo pipeline

- **Capture:** support camera/gallery and orientation correction. Proposed limits: five photos per catch, 15 MB source per image, 4 MB uploaded result and 20 megapixels decoded. These are configurable product defaults, not platform limits
- **Prepare:** resize/compress in the client, remove EXIF/GPS metadata and show progress. A server-side validation/sanitization step must enforce safe image type, dimensions and metadata removal for untrusted uploads. Select a runtime-compatible decoder or approved processing service; unsupported formats get a useful conversion message
- **Store:** accept JPEG, PNG and WebP after checking actual content, not filename alone. Reject SVG/HTML and mismatched types. Use generated storage keys with no user-controlled paths. Authorize the catch before receiving/storing the image
- **Display:** fetch through the authenticated API, then render a blob URL if using bearer tokens. Do not put tokens in URLs or rely on a normal img tag to add an Authorization header. Revoke local object URLs when no longer needed
- **Recover:** save catch data separately from photo processing. Mark individual photos failed/pending/ready and allow retry. Use a pending record plus retryable cleanup for partial DB/R2 failures; never claim a cross-service transaction

Privacy and resilience

Tenant isolation and security controls

- Centralize identity verification and owner-scoped database access. Check each parent-child relationship on creation and reassignment. Opaque IDs improve hygiene but never replace authorization
- Use TLS, restrictive CORS for approved origins, a Content Security Policy, safe output encoding and content-type/nosniff headers. If switching to cookie sessions, design HttpOnly/Secure/SameSite cookies and CSRF defenses explicitly; do not mix authentication patterns casually
- Send private API and image responses with private, no-store caching; shared content follows page 8 visibility rules. Exclude /api/* and auth-helper paths from service-worker caches and SPA fallbacks. Ensure protected-route failures fail closed
- Keep service credentials in server-side secret storage. Never ship R2/D1 credentials or an admin key to the browser. Firebase web configuration identifies the app; it does not grant data authorization
- Log request IDs, safe error codes and aggregate operational metrics. Redact bearer tokens, image bytes, exact venue coordinates, notes and unnecessary personal identifiers

Poor connectivity without false success

Release 1 remains online-first. Preserve unsubmitted forms and selected photo state where the browser supports it. A disconnected save stays visibly Unsent or Failed; only server acknowledgement produces Saved. Namespace persistent drafts by UID, avoid access from another signed-in account, and explain local storage on a shared device.

On sign-out, warn about unsent drafts and offer an explicit keep/discard decision before clearing account-scoped caches. After authentication expires, preserve the draft, sign in again, then retry with the same idempotency key. Do not silently upload another account's drafts. Browser eviction can remove local drafts, so do not describe them as a durable backup.

Deletion and export

Confirm destructive actions with their scope: deleting a session removes its catches and photos. Use cascading relational cleanup plus a durable, retryable R2 deletion workflow; hide deleted records immediately and report deletion-pending states truthfully. Account deletion first blocks access, then removes owned data and the auth account with retry/recovery handling. Export only the requester's data in a documented JSON/CSV format, with an owned-photo download option.

Publish a short privacy notice describing Google/Firebase sign-in, Cloudflare storage, location/photo handling, retention, account deletion and a contact route. Decide backup retention and deletion timelines before public launch. Do not expose precise fishing venues through analytics or URLs. Shared views and map responses may reveal only the location precision explicitly selected by the poster (page 8).

Capacity and future offline work

Open sign-up needs per-user and IP request limits, bounded queries, upload/storage quotas and abuse controls. Set configurable limits and budget alerts; notifications alone are not a hard spending cap. Do not promise the service will be free forever.

Phase 2 can add an IndexedDB queue for records and photo blobs, explicit draft/uploading/synced states, exponential retry, resumable recovery and conflict handling. Full offline support requires dedicated multi-device and low-connectivity testing; a cached app shell is not enough.

Build order and acceptance tests

Implementation milestones

- 1. Prove the foundation.** Scaffold the client and Pages Functions, configure local D1/R2 bindings, create migrations, and test real Google sign-in on a stable staging domain. Demonstrate user A cannot read user B's test record or image.
- 2. Complete the logging loop.** Build session/catch CRUD, weights, private photos, summaries and history, then the Google Maps feed and sharing controls. Add idempotency, drafts and explicit upload recovery as part of the workflow, not as final polish.
- 3. Make it launchable.** Add export/deletion, reporting/moderation, quotas, privacy copy, accessibility and error reporting and clean staging/production separation. Run the checks below with two independent Google accounts and real phones.

Test	Pass condition
Identity	Fresh external Google user signs in; cancelled login is recoverable. Refresh, sign out and sign in work on mobile and installed PWA.
Token validation	Missing, expired, malformed, wrong-audience and bad-signature tokens fail. Disabled/deleting app users fail. Revocation behavior matches the documented policy.
Cross-user access	A cannot list, read, edit, delete or export B's sessions/catches/locations/photos or attach a child to B's record, including guessed IDs and direct photo requests.
Session lifecycle	Start/edit/end works. End cannot precede start; historical entries are possible. Zero-catch sessions are saved. Duplicate taps create one record.
Catch validation	Actual species can differ from targets. Unweighed stays NULL. kg and lb/oz round trips preserve values. Invalid weights/times are rejected with field errors.
Photo security	Valid image uploads work; oversized, disguised and unsupported files fail safely. GPS metadata is removed. Private content is inaccessible to other users; shared media follows post visibility and revocation rules.
Failure recovery	Lost network, auth expiry, failed upload and lost response preserve the catch/draft. Retry does not duplicate a catch or photo. Conflicting edits are visible.
History and statistics	Venue/species/date filters compose correctly. Edits/deletes update totals and bests. Unweighed catches never become a zero-weight personal best.
Deletion and export	Export contains only the user's data. Session/account deletion hides records, removes blobs through retries and accurately reports any pending cleanup.
Usability	Keyboard and screen-reader labels work; focus remains visible; 200% zoom is usable. Test narrow screens, touch controls and slow mobile connections.
Sharing and map	Unselected fields are absent from responses. Approximate pins never leak exact coordinates. Unshare/delete removes the pin, card and media access. Reports reach moderation.
Deploy and operations	Preview uses isolated data. Migrations, deep links, API 404s, rollback and cleanup retries work.

Definition of done: every confirmed requirement and the agreed MVP work end to end; the test results above are recorded; setup, secrets, deploy, rollback and known limitations are documented. An attractive mockup alone is not a completed release.

Social map and selective sharing

New confirmed scope: show other people's catches on Google Maps with photo pins. The poster chooses whether to keep a catch private and which details to share. A shared post is a separate, limited representation of a private catch.

Sharing controls and audience

Default every catch to Private. Offer a Share catch screen with switches for species, weight, bait/lure, date/time, venue name, notes and selected media. Show the exact shared-card preview and audience before Publish. Recommended MVP audience: other signed-in Keepnet users; anonymous public browsing is a separate launch decision. Use an optional display alias; never publish Google email or automatically reuse a Google profile photo.

Location has an independent Hidden / Approximate / Exact choice. Hidden means no map pin; the shared card can still exist in the list. Approximate uses a stable coarse grid or area center with a displayed radius, such as 1 km. Exact requires a clear warning and explicit selection. Venue name, peg, timestamps and captions can reveal a spot too: keep them separate and warn before sharing identifying details.

Projection model and map API

Post: id, owner_uid, catch_id, visibility, audience, shared_fields, selected_media_ids, public_alias, location_precision, published_coordinates, radius_m, published_at, updated_at, version, moderation_state.

SharedMedia: post_id, source_media_id, sanitized_derivative_key, status. **Report:** reporter_uid, post_id, reason, state, created_at. Add a block list and tightly controlled moderator role.

POST/PATCH/DELETE /posts manages only the owner's post. GET /posts/map accepts bounded viewport, zoom and cursor filters, returns clustered approved pins, and never searches private coordinates. GET /posts/:id and shared-media reads use the same audience/visibility check. POST /posts/:id/reports and /blocks create abuse reports and user blocks. Rate-limit all routes.

Construct a fresh server-side allowlisted response from the sharing settings; never send the whole catch and hide fields in the UI. Index and filter only published coordinates and approved shared fields. Use the same rules for thumbnails, clusters, bounding-box queries, exports and previews. The original photo, EXIF/GPS data and private relation IDs must not leak.

Unsharing and moderation

Editing a private catch must not silently add newly filled fields to its post. Refresh only fields already approved, with a preview when expanding disclosure. On unshare/delete, revoke card and media access, remove map/index entries and invalidate caches. Prefer visibility-checked media responses and no-store for MVP. Any future signed URLs/cache TTLs need a documented revocation window. Downloaded copies and screenshots cannot be recalled.

Include Report post, block-user filtering, basic community rules and a minimal moderator queue with hide/remove/suspend actions and an audit trail. Avoid comments, direct messages and followers in the first release.

Optional short videos and Maps setup

Phase 2 recommendation: one 15-30 second clip per catch, validated size/duration/codecs, metadata stripping, safe transcoding, poster image and muted click-to-play. Keep private by default and separately selectable for sharing. R2 storage alone is not a video-processing pipeline; choose and cost a compatible service before enabling uploads.

Production Google Maps requires billing and a standard API key. Restrict the browser key to approved HTTP referrers and required APIs; it is visible in browser requests, not a server secret. Keep server-side keys separate. Set quotas and budget alerts; approve billing before activation. Retain required attribution. [9, 10]

Deployment handoff and references

Repository and delivery checklist

- Deliver source, lockfile, README, database migrations, seeded demo fixtures, shared validation schemas, automated tests and an environment-variable example containing placeholders only
- Suggested layout: src/ for UI, functions/api/ for request handlers, shared/ for schemas/types, migrations/ for SQL, tests/ for unit/integration/end-to-end checks. Keep authorization and media processing in small reusable server modules
- Use version-controlled Wrangler configuration for the Pages output directory, compatibility date and DB/PHOTOS bindings. Separate local, staging and production resources; previews must never point to production data. Verify configuration against current Pages docs. [2, 7]
- Document the build command/output directory, Pages Functions route inclusion, frontend deep-link fallback, auth-helper routing, secret setup and migration order. Keep /api/* out of HTML fallback handling
- Provide a staging URL only after authorized deployment, a test account/setup guide without sharing passwords, screenshots, the acceptance-test report and a list of unresolved limitations

Owner decisions before public launch

Confirm the production domain and account owners, deployment approval, service budget, photo/storage quotas, privacy/contact details, retention/deletion policy and the revocation implementation. Approve Google Maps billing, the sharing audience and moderation owner; identify image/video processing costs before enabling those services. Obtain explicit approval before a paid upgrade, new persistent credential, production access change or public deployment.

Pages Functions use Workers pricing; database, storage and other services have their own usage considerations. Recheck current pricing at implementation time and document a small usage-based estimate rather than assuming zero cost. [8]

Official implementation references

Documentation checked 3 October 2026. These links support the platform and authentication details; the product defaults and architecture choices in this brief are recommendations. Recheck current docs before implementation.

- [1] [Cloudflare Pages Functions](#)
- [2] [Pages Functions bindings for D1 and R2](#)
- [3] [R2 public access controls](#)
- [4] [Firebase Google sign-in for web](#)
- [5] [Firebase ID-token verification and revocation caveat](#)
- [6] [Firebase redirect login and third-party storage](#)
- [7] [Pages Functions Wrangler configuration](#)
- [8] [Pages Functions pricing](#)
- [9] [Google Maps JavaScript setup and production billing](#)
- [10] [Google Maps API key restrictions and security](#)

UI mockups are a separate visual reference. The written requirements and privacy rules in this updated brief take precedence over visual shorthand.